
```
>>> numbers = ['1', '2', '3', '4', '5']
>>> for i, number in enumerate(numbers):
...     numbers[i] = int(number)
...
>>> numbers
[1, 2, 3, 4, 5]
```

A visualization of the execution of this code is at <https://autbor.com/covertstringnumbers>. Modifying the items in the list is fine; it's changing the number of items in the list that is bug prone.

Another possible way to add or delete items in a list safely is by iterating backward from the end of the list to the beginning. This way, you can delete items from the list as you iterate over it, or add items to the list as long as you add them to the end of the list. For example, enter the following code, which removes even integers from the `someInts` list.

```
>>> someInts = [1, 7, 4, 5]
>>> for i in range(len(someInts)):
...
...     if someInts[i] % 2 == 0:
...         del someInts[i]
...
Traceback (most recent call last):
  File "<stdin>", line 2, in <module>
IndexError: list index out of range
>>> someInts = [1, 7, 4, 5]
>>> for i in range(len(someInts) - 1, -1, -1):
...     if someInts[i] % 2 == 0:
...         del someInts[i]
...
>>> someInts
[1, 7, 5]
```

This code works because none of the items that the loop will iterate over in the future ever have their index changed. But the repeated shifting up of values after the deleted value makes this technique inefficient for long lists. A visualization of the execution of this code is at <https://autbor.com/iteratebackwards1>. You can see the difference between iterating forward and backward in [Figure 8-3](#).